



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

K. J. Somaiya School of Engineering, Mumbai-77
(Somaiya Vidyavihar University)
Department of Computer Engineering



Batch: 1

Roll No.: 16010123012

Experiment / Assignment / Tutorial No.: 1

Title: Data Preprocessing and feature engineering in machine learning.

Aim: To understand & implement various data preprocessing and feature engineering techniques using Python for machine learning applications.

Expected Outcome of Experiment:

CO1: To establish a strong foundation in machine learning fundamentals.

Books/ Journals/ Websites referred:

1. <https://www.python.org/about/>
2. <https://www.upgrad.com/blog/data-preprocessing-in-machine-learning/>
3. <https://towardsdatascience.com/data-preprocessing-concepts-fa946d11c825>
4. Data Mining: Concepts and Techniques by Jiawei Han, Micheline Kamber, Jian Pei

Introduction

What is a feature and why do we need the engineering of it? Basically, all machine learning algorithms use some input data to create outputs. This input data comprise features, which are usually in the form of structured columns. Algorithms require features with some specific characteristics to work properly. Here, the need for feature engineering arises. Feature engineering efforts mainly have two goals:

- Preparing the proper input dataset which is compatible with the machine learning algorithm requirements.
- Improving the performance of machine learning models.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

K. J. Somaiya School of Engineering, Mumbai-77
(Somaiya Vidyavihar University)
Department of Computer Engineering



Different Preprocessing Techniques:

1. Imputation
2. Handling Outliers
3. Binning
4. Log Transform
5. One-Hot Encoding
6. Grouping Operations
7. Feature Split
8. Scaling

**** For a given dataset (housing.csv), implement the above preprocessing techniques, explain & attach the necessary snapshots for function/library used.**

1. Imputation

Imputing is the process of filling in missing data in a dataset. There are several techniques for imputing missing values in a dataset, such as mean imputation, median imputation, mode imputation, and KNN imputation.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import kagglehub
from kagglehub import KaggleDatasetAdapter

file_path = "housing.csv"

# Load the latest version
df = kagglehub.load_dataset(
    KaggleDatasetAdapter.PANDAS,
    "camnugent/california-housing-prices",
    file_path,
)

print("First 5 records:", df.head())

*** /tmp/ipython-input-965094521.py:7: DeprecationWarning: Use dataset_load() instead of load_dataset(). load_dataset() will be removed in a future version.
    df = kagglehub.load_dataset(
Using Colab cache for faster access to the 'california-housing-prices' dataset.
First 5 records:
  longitude  latitude  housing_median_age  total_rooms  total_bedrooms \
0   -122.23    37.88           41.0           880.0           129.0
1   -122.22    37.86           21.0          7099.0          1106.0
2   -122.24    37.85           52.0          1467.0           190.0
3   -122.25    37.85           52.0          1274.0           235.0
4   -122.25    37.85           52.0          1627.0           280.0

  population  households  median_income  median_house_value  ocean_proximity
0         322.0         126.0         8.3252         452600.0         NEAR BAY
1        2401.0        1130.0         8.3014        358500.0         NEAR BAY
2         496.0         177.0         7.2574        352100.0         NEAR BAY
3         558.0         219.0         5.6431        341300.0         NEAR BAY
4         565.0         250.0         3.8462        342200.0         NEAR BAY
```

```

null_values = df.isnull().any()
print(f"Null Values:\n{null_values}")

total_null = df.isnull().sum()
print(f"\nTotal Null Values:\n{total_null}")

```

```

... Null Values:
longitude      False
latitude       False
housing_median_age  False
total_rooms    False
total_bedrooms  True
population     False
households     False
median_income  False
median_house_value  False
ocean_proximity  False
dtype: bool

```

```

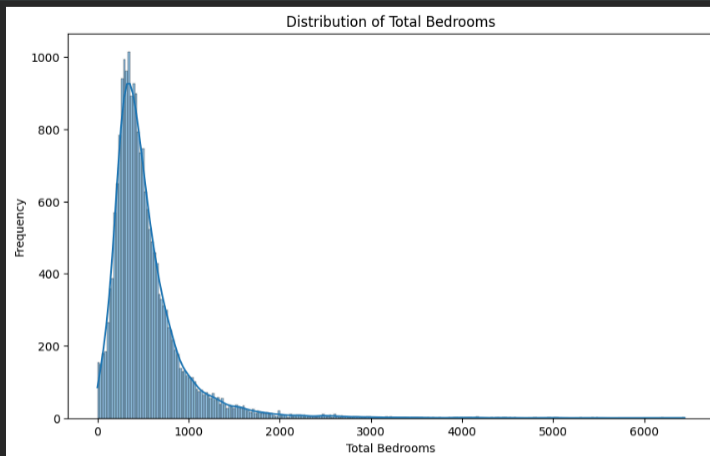
Total Null Values:
longitude      0
latitude       0
housing_median_age  0
total_rooms    0
total_bedrooms  207
population     0
households     0
median_income  0
median_house_value  0
ocean_proximity  0
dtype: int64

```

```

plt.figure(figsize=(10, 6))
sns.histplot(df['total_bedrooms'], kde=True)
plt.title('Distribution of Total Bedrooms')
plt.xlabel('Total Bedrooms')
plt.ylabel('Frequency')
plt.show()

```



```

median_total_bedrooms = df['total_bedrooms'].median()
df['total_bedrooms'] = df['total_bedrooms'].fillna(median_total_bedrooms)

print(f"Null values after filling:\n{df.isnull().sum()}")

```

```

Null values after filling:
longitude      0
latitude       0
housing_median_age  0
total_rooms    0
total_bedrooms  0
population     0
households     0
median_income  0
median_house_value  0
ocean_proximity  0

```

2. Handling Outliers

Handling outliers is a data pre-processing technique that involves identifying and dealing with data points that are significantly different from the rest of the dataset. Outliers can occur due to measurement errors, data entry errors, or simply because they represent extreme values. Handling outliers is important because they can skew the results of data analysis and machine learning models. There are several techniques for handling outliers, including removing them, Winsorizing the data, transforming the data, or leaving them as is and using robust statistical methods that are less sensitive to outliers. The specific technique used will depend on the nature of the data and the goals of the analysis.

Visualize the data: Before you can handle outliers, you need to identify them. One way to do this is by visualizing the data using box plots, histograms, or scatter plots. Outliers are typically data points that are far away from the rest of the data, so they will be visible as points that are outside the range of the majority of the data.

Remove outliers: If you have a small number of outliers in your dataset, you can consider removing them. One way to do this is by using the Z-score method. You can calculate the Z-score for each data point, and remove any data point that has a Z-score greater than a certain threshold (e.g. 3). Here's an example:

```
numerical_cols = df.select_dtypes(include=[np.number]).columns.tolist()

print("Numerical columns identified for outlier detection: ")
print(numerical_cols)

Numerical columns identified for outlier detection:
['longitude', 'latitude', 'housing_median_age', 'total_rooms', 'total_bedrooms', 'population', 'households', 'median_income', 'median_house_value']

outlier_info = {}

for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers_count = df[(df[col] < lower_bound) | (df[col] > upper_bound)].shape[0]

    outlier_info[col] = {
        'Q1': Q1,
        'Q3': Q3,
        'IQR': IQR,
        'lower_bound': lower_bound,
        'upper_bound': upper_bound,
        'outliers_count': outliers_count
    }

print(f"\n--- Column: {col} ---")
print(f"Q1 (25th percentile): {Q1:.2f}")
print(f"Q3 (75th percentile): {Q3:.2f}")
print(f"IQR: {IQR:.2f}")
print(f"Lower Bound for Outliers: {lower_bound:.2f}")
print(f"Upper Bound for Outliers: {upper_bound:.2f}")
print(f"Number of outliers detected: {outliers_count}")

***
--- Column: longitude ---
Q1 (25th percentile): -121.80
Q3 (75th percentile): -118.01
IQR: 3.79
Lower Bound for Outliers: -127.48
Upper Bound for Outliers: -112.33
Number of outliers detected: 0

--- Column: latitude ---
Q1 (25th percentile): 33.93
Q3 (75th percentile): 37.71
IQR: 3.78
Lower Bound for Outliers: 28.26
Upper Bound for Outliers: 43.38
Number of outliers detected: 0
```



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

K. J. Somaiya School of Engineering, Mumbai-77
(Somaiya Vidyavihar University)
Department of Computer Engineering



```
--- Column: housing_median_age ---
***
Q1 (25th percentile): 18.00
Q3 (75th percentile): 37.00
IQR: 19.00
Lower Bound for Outliers: -10.50
Upper Bound for Outliers: 65.50
Number of outliers detected: 0

--- Column: total_rooms ---
Q1 (25th percentile): 1447.75
Q3 (75th percentile): 3148.00
IQR: 1700.25
Lower Bound for Outliers: -1102.62
Upper Bound for Outliers: 5698.38
Number of outliers detected: 1287

--- Column: total_bedrooms ---
Q1 (25th percentile): 297.00
Q3 (75th percentile): 643.25
IQR: 346.25
Lower Bound for Outliers: -222.38
Upper Bound for Outliers: 1162.62
Number of outliers detected: 1306

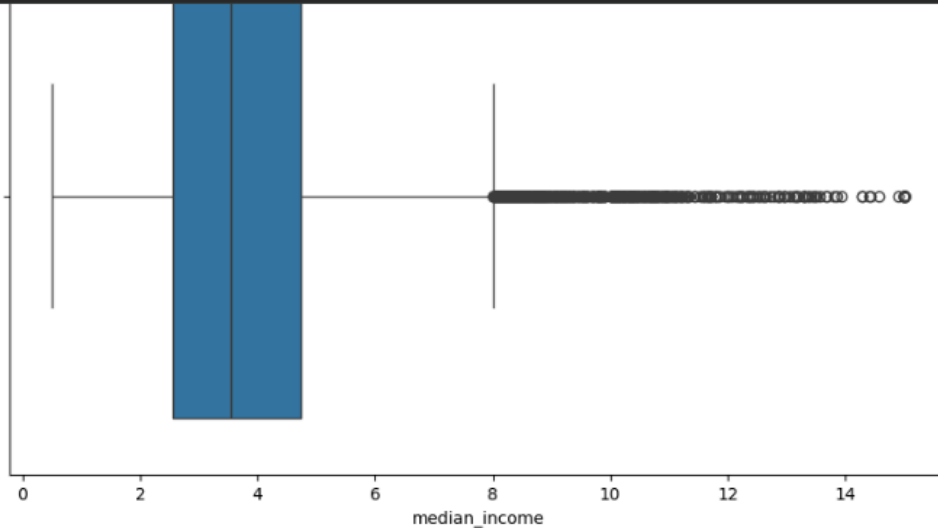
--- Column: population ---
Q1 (25th percentile): 787.00
Q3 (75th percentile): 1725.00
IQR: 938.00
Lower Bound for Outliers: -620.00
Upper Bound for Outliers: 3132.00
Number of outliers detected: 1196

--- Column: households ---
Q1 (25th percentile): 280.00
Q3 (75th percentile): 605.00
IQR: 325.00
Lower Bound for Outliers: -207.50
Upper Bound for Outliers: 1092.50
Number of outliers detected: 1220

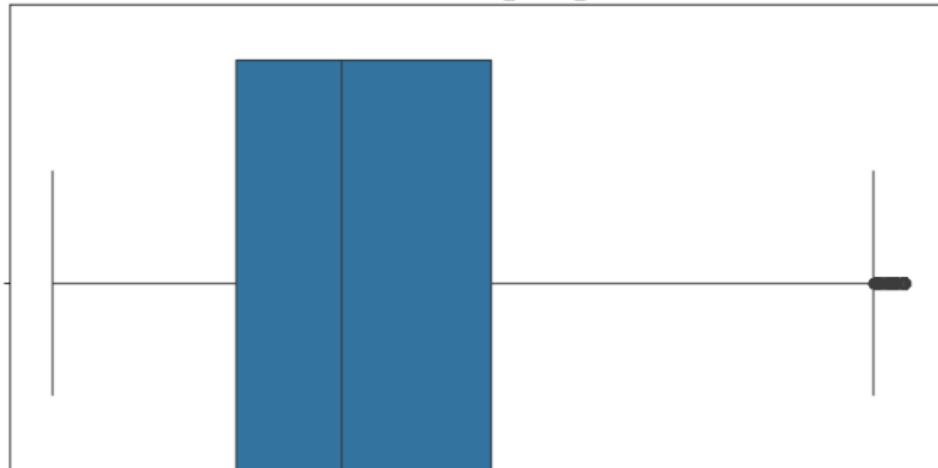
--- Column: median_income ---
Q1 (25th percentile): 2.56
Q3 (75th percentile): 4.74
IQR: 2.18
Lower Bound for Outliers: -0.71
Upper Bound for Outliers: 8.01
Number of outliers detected: 681

--- Column: median_house_value ---
Q1 (25th percentile): 119600.00
Q3 (75th percentile): 264725.00
IQR: 145125.00
Lower Bound for Outliers: -98087.50
Upper Bound for Outliers: 482412.50
Number of outliers detected: 1071
```

```
for col in numerical_cols:
    plt.figure(figsize=(10, 6))
    sns.boxplot(x=df[col])
    plt.title(f'Box Plot of {col}')
    plt.xlabel(col)
    plt.show()
```



Box Plot of median_house_value



```
print("Outlier Summary:")
for col, info in outlier_info.items():
    if info['outliers_count'] > 0:
        print(f"- Column '{col}': {info['outliers_count']} outliers detected. (Lower Bound: {info['lower_bound']:.2f}, Upper Bound: {info['upper_bound']:.2f})")
    else:
        print(f"- Column '{col}': No outliers detected within the 1.5*IQR range.")
```

Outlier Summary:

- Column 'longitude': No outliers detected within the 1.5*IQR range.
- Column 'latitude': No outliers detected within the 1.5*IQR range.
- Column 'housing_median_age': No outliers detected within the 1.5*IQR range.
- Column 'total_rooms': 1287 outliers detected. (Lower Bound: -1102.62, Upper Bound: 5698.38)
- Column 'total_bedrooms': 1306 outliers detected. (Lower Bound: -222.38, Upper Bound: 1162.62)
- Column 'population': 1196 outliers detected. (Lower Bound: -620.00, Upper Bound: 3132.00)
- Column 'households': 1220 outliers detected. (Lower Bound: -207.50, Upper Bound: 1092.50)
- Column 'median_income': 681 outliers detected. (Lower Bound: -0.71, Upper Bound: 8.01)
- Column 'median_house_value': 1071 outliers detected. (Lower Bound: -98087.50, Upper Bound: 482412.50)

3. Binning and One-Hot Coding

Binning is a common data pre-processing technique used to transform continuous numerical data into discrete categories or bins.

```
import pandas as pd

bins = [0, 15, 30, 45, 60, 100]
labels = ['New', 'Moderately New', 'Established', 'Old', 'Very Old']
df['housing_median_age_binned'] = pd.cut(df['housing_median_age'], bins=bins, labels=labels, right=False)

print("Value counts for 'housing_median_age_binned':")
print(df['housing_median_age_binned'].value_counts())
print("\nFirst 5 records with binned housing_median_age:")
print(df[['housing_median_age', 'housing_median_age_binned']].head())
```

```
*** Value counts for 'housing_median_age_binned':
housing_median_age_binned
Moderately New    7894
Established       7466
New               2775
Old               2505
Very Old          0
Name: count, dtype: int64

First 5 records with binned housing_median_age:
housing_median_age housing_median_age_binned
0                41.0      Established
1                21.0    Moderately New
2                52.0         Old
3                52.0         Old
4                52.0         Old
```

```
df = pd.get_dummies(df, columns=['ocean_proximity'], prefix='ocean_proximity')

print("First 5 records after one-hot encoding:")
print(df.head())
```

```
*** First 5 records after one-hot encoding:
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
0	-122.23	37.88	41.0	880.000	129.0	
1	-122.22	37.86	21.0	5698.375	1106.0	
2	-122.24	37.85	52.0	1467.000	190.0	
3	-122.25	37.85	52.0	1274.000	235.0	
4	-122.25	37.85	52.0	1627.000	280.0	

	population	households	median_income	median_house_value	\
0	322.0	126.0	2.232720	452600.0	
1	2401.0	1092.5	2.230165	358500.0	
2	496.0	177.0	2.111110	352100.0	
3	558.0	219.0	1.893579	341300.0	
4	565.0	259.0	1.578195	342200.0	

	rooms_per_household	bedrooms_per_room	population_per_household	\
0	6.984127	0.146591	2.555556	
1	5.215904	0.194090	2.197712	
2	8.288136	0.129516	2.802260	
3	5.817352	0.184458	2.547945	
4	6.281853	0.172096	2.181467	

	housing_median_age_binned	ocean_proximity_<1H	OCEAN	\
0	Established		False	
1	Moderately New		False	
2	Old		False	
3	Old		False	
4	Old		False	

	ocean_proximity_INLAND	ocean_proximity_ISLAND	ocean_proximity_NEAR	BAY	\
0	False	False		True	
1	False	False		True	
2	False	False		True	
3	False	False		True	
4	False	False		True	

	ocean_proximity_NEAR	OCEAN
0	False	
1	False	
2	False	
3	False	
4	False	

4. Log Transform

We first load our dataset into a Pandas DataFrame using the `read_csv` function. Then, we perform a log transformation on the 'Price' column using the `log` function from the `numpy` library. Finally, we print the first few rows of the transformed dataset using the `head` method.

```
import numpy as np

winsorize_cols = ['total_rooms', 'total_bedrooms', 'population', 'households', 'median_house_value']

for col in winsorize_cols:
    if col in outlier_info:
        lower_bound = outlier_info[col]['lower_bound']
        upper_bound = outlier_info[col]['upper_bound']
        df[col] = np.clip(df[col], lower_bound, upper_bound)
        print(f"Column '{col}' Winsorized using bounds ({lower_bound:.2f}, {upper_bound:.2f}).")
    else:
        print(f"Warning: Outlier info not found for column '{col}'. Skipping Winsorization.")

log_transform_col = 'median_income'
if log_transform_col in df.columns:
    df[log_transform_col] = np.log1p(df[log_transform_col])
    print(f"Column '{log_transform_col}' underwent Logarithmic Transformation.")
else:
    print(f"Warning: Column '{log_transform_col}' not found. Skipping Logarithmic Transformation.")

print(f"\nUpdated DataFrame shape: {df.shape}")

Column 'total_rooms' Winsorized using bounds (-1102.62, 5698.38).
Column 'total_bedrooms' Winsorized using bounds (-222.38, 1162.62).
Column 'population' Winsorized using bounds (-620.00, 3132.00).
Column 'households' Winsorized using bounds (-207.50, 1092.50).
Column 'median_house_value' Winsorized using bounds (-98087.50, 482412.50).
Column 'median_income' underwent Logarithmic Transformation.

Updated DataFrame shape: (20640, 18)
```

5. Grouping Operations:

```
ocean_proximity_categories = [col.replace('ocean_proximity_', '') for col in df.columns if col.startswith('ocean_proximity_')]

print("Mean median_house_value by ocean_proximity category:")
mean_house_value_by_proximity = {}
for category in ocean_proximity_categories:
    col_name = f'ocean_proximity_{category}'
    mean_value = df[df[col_name] == True]['median_house_value'].mean()
    mean_house_value_by_proximity[category] = mean_value
    print(f" {category}: {mean_value:.2f}")

*** Mean median_house_value by ocean_proximity category:
<1H OCEAN: 239004.44
INLAND: 124728.61
ISLAND: 380440.00
NEAR BAY: 257614.54
NEAR OCEAN: 247919.97
```

6. Feature Split

```
df['rooms_per_household'] = df['total_rooms'] / df['households']
df['bedrooms_per_room'] = df['total_bedrooms'] / df['total_rooms']
df['population_per_household'] = df['population'] / df['households']

print("First 5 records with new features:")
print(df.head())
```

*** First 5 records with new features:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
0	-122.23	37.88	41.0	880.000	129.0	
1	-122.22	37.86	21.0	5698.375	1106.0	
2	-122.24	37.85	52.0	1467.000	190.0	
3	-122.25	37.85	52.0	1274.000	235.0	
4	-122.25	37.85	52.0	1627.000	280.0	

	population	households	median_income	median_house_value	ocean_proximity	\
0	322.0	126.0	2.232720	452600.0	NEAR BAY	
1	2401.0	1092.5	2.230165	358500.0	NEAR BAY	
2	496.0	177.0	2.111110	352100.0	NEAR BAY	
3	558.0	219.0	1.893579	341300.0	NEAR BAY	
4	565.0	259.0	1.578195	342200.0	NEAR BAY	

	rooms_per_household	bedrooms_per_room	population_per_household
0	6.984127	0.146591	2.555556
1	5.215904	0.194090	2.197712
2	8.288136	0.129516	2.802260
3	5.817352	0.184458	2.547945
4	6.281853	0.172096	2.181467

7. Scaling

First load the 'housing.csv' dataset using `pd.read_csv()`. Then, we initialize a `MinMaxScaler` object and select the columns to scale. Then fit and transform the selected columns using the `fit_transform()` method of the scaler and update the dataset with the scaled values. Finally, printed the head of the scaled dataset using `print(df.head())`.

```
df = df.drop('housing_median_age', axis=1)

numerical_cols_to_scale = df.select_dtypes(include=[np.number]).columns.tolist()
if 'median_house_value' in numerical_cols_to_scale:
    numerical_cols_to_scale.remove('median_house_value')

print("Numerical columns to be scaled:")
print(numerical_cols_to_scale)

scaler = StandardScaler()

df[numerical_cols_to_scale] = scaler.fit_transform(df[numerical_cols_to_scale])

print("\nFirst 5 records after scaling numerical features:")
print(df.head())
```

```
*** Numerical columns to be scaled:
['longitude', 'latitude', 'total_rooms', 'total_bedrooms', 'population', 'households', 'median_income', 'rooms_per_household', 'bedrooms_per_room', 'population_per_household']

First 5 records after scaling numerical features:
  longitude  latitude  total_rooms  total_bedrooms  population  households \
0  -1.327835  1.052548   -1.117285   -1.309916   -1.325821   -1.291972
1  -1.322844  1.043185    2.329936    2.128690    1.389936    2.348314
2  -1.332827  1.038503   -0.697327   -1.095223   -1.098528   -1.099883
3  -1.337818  1.038503   -0.835405   -0.936843   -1.017539   -0.941691
4  -1.337818  1.038503   -0.582857   -0.778463   -1.008395   -0.791033

  median_income  median_house_value  rooms_per_household  bedrooms_per_room \
0    1.808451    452680.0         0.660783         -1.057145
1    1.802962    358500.0        -0.074654         -0.307692
2    1.542276    352100.0         1.203144         -1.326555
3    1.039104    341300.0         0.175499         -0.459668
4     0.238002    342200.0         0.368694         -0.654725

  population_per_household  housing_median_age_binned \
0          -0.085602          Established
1          -0.154411          Moderately New
2          -0.038164             Old
3          -0.067066             Old
4          -0.157534             Old

  ocean_proximity_1H OCEAN  ocean_proximity_INLAND  ocean_proximity_ISLAND \
0             False      False                  False
1             False      False                  False
2             False      False                  False
3             False      False                  False
4             False      False                  False

  ocean_proximity_NEAR BAY  ocean_proximity_NEAR OCEAN
0              True      False
1              True      False
2              True      False
3              True      False
4              True      False
```

```
print("\nDescriptive Statistics of Transformed DataFrame:")
print(df.describe())

print("\nSample of Transformed DataFrame (first 5 rows):")
print(df.head())
```

```
***
Descriptive Statistics of Transformed DataFrame:
count    longitude    latitude    total_rooms    total_bedrooms    population  \
count    2.064000e+04    2.064000e+04    2.064000e+04    2.064000e+04    2.064000e+04
mean     -8.526513e-15    -1.079584e-15    -7.711317e-17    4.131062e-17    1.335710e-16
std       1.000024e+00    1.000024e+00    1.000024e+00    1.000024e+00    1.000024e+00
min      -2.385992e+00    -1.447568e+00    -1.745435e+00    -1.760419e+00    -1.742525e+00
25%      -1.113209e+00    -7.967887e-01    -7.110986e-01    -7.186303e-01    -7.184008e-01
50%       5.389137e-01    -6.422871e-01    -2.251412e-01    -2.329315e-01    -2.233204e-01
75%      7.784964e-01    9.729566e-01    5.053153e-01    5.000160e-01    5.068904e-01
max       2.625280e+00    2.958068e+00    2.329936e+00    2.327985e+00    2.344827e+00

count    households    median_income    median_house_value    rooms_per_household  \
count    2.064000e+04    2.064000e+04    20640.000000    2.064000e+04
mean     -6.058892e-17    1.321940e-16    205981.224976    2.698961e-16
std       1.000024e+00    1.000024e+00    113217.350152    1.000024e+00
min      -1.762780e+00    -3.973998e+00    14999.000000    -1.892114e+00
25%      -7.119372e-01    -6.435560e-01    119600.000000    -3.778157e-01
50%      -2.260635e-01    5.684127e-02    179700.000000    -7.465398e-02
75%      5.121632e-01    6.808332e-01    264725.000000    2.354831e-01
max       2.348314e+00    2.880519e+00    482412.500000    5.677856e+01

count    bedrooms_per_room    population_per_household
count    2.064000e+04    2.064000e+04
mean     -4.186143e-16    7.917870e-17
std       1.000024e+00    1.000024e+00
min      -2.165611e+00    -4.438783e-01
25%      -5.679672e-01    -1.055498e-01
50%      -1.509046e-01    -2.911878e-02
75%      3.753015e-01    4.757542e-02
max       4.119798e+01    9.979619e+01

Sample of Transformed DataFrame (first 5 rows):
count    longitude    latitude    total_rooms    total_bedrooms    population    households  \
0    -1.327835    1.052548    -1.117285    -1.309916    -1.325821    -1.291972
1    -1.322844    1.043185    2.329936    2.128690    1.389936    2.348314
2    -1.332827    1.038503    -0.697327    -1.095223    -1.098528    -1.099883
3    -1.337818    1.038503    -0.835405    -0.936843    -1.017539    -0.941691
4    -1.337818    1.038503    -0.582857    -0.778463    -1.008395    -0.791033

count    median_income    median_house_value    rooms_per_household    bedrooms_per_room  \
0    1.808451    452600.0    0.660783    -1.057145
1    1.802962    358500.0    -0.074654    -0.307692
2    1.542276    352100.0    1.203144    -1.326555
3    1.039104    341300.0    0.175499    -0.459668
4    0.238002    342200.0    0.368694    -0.654725

count    population_per_household    housing_median_age_binned  \
0    -0.085602    Established
1    -0.154411    Moderately New
2    -0.038164    Old
```

Conclusion:

This experiment successfully demonstrated the importance of data preprocessing and feature engineering as a foundational step in the machine learning pipeline. Using the California Housing dataset, various preprocessing techniques were applied to convert raw, real-world data into a structured and model-ready form. Missing values were handled using imputation to prevent data loss and ensure numerical stability. Outlier handling techniques were discussed to reduce skew and improve model robustness. Binning and log transformation helped simplify complex numerical distributions and reduce the impact of extreme values. One-hot encoding enabled categorical variables to be represented numerically without introducing false ordinal relationships. Grouping operations and feature splitting created more informative features by



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

K. J. Somaiya School of Engineering, Mumbai-77
(Somaiya Vidyavihar University)

Department of Computer Engineering



extracting meaningful patterns from existing data. Finally, feature scaling standardized numerical ranges, ensuring fair contribution of all features during model training.

Post lab Questions:

Explain following text Pre-processing techniques with example:

- **Tokenization**
Tokenization is the process of breaking raw text into smaller units called tokens, which are usually words, sentences, or characters. It is the first and most essential step in text preprocessing because machine learning models cannot directly understand continuous text strings. By tokenizing text, we convert human language into manageable pieces that can later be transformed into numerical features. For example, the sentence “Machine learning is powerful” when tokenized at the word level becomes [“Machine”, “learning”, “is”, “powerful”]. Tokenization helps models analyze text patterns, word frequencies, and relationships between words in tasks such as sentiment analysis, text classification, and information retrieval.
- **Stemming**
Stemming is a preprocessing technique used to reduce words to their root or base form by removing suffixes using simple rules. The goal of stemming is to group together different forms of the same word so they are treated as a single feature by the model. For example, the words “playing”, “played”, and “plays” are all reduced to the stem “play”. This reduces vocabulary size and improves computational efficiency. However, stemming can sometimes produce words that are not grammatically correct, such as converting “studies” to “studi”. Despite this limitation, stemming is widely used in applications where speed is more important than linguistic accuracy, such as search engines.
- **Lemmatization**
Lemmatization is a more advanced preprocessing technique that converts words into their base or dictionary form, known as a lemma, by considering grammatical rules and context. Unlike stemming, lemmatization always produces valid and meaningful words. For example, “running” becomes “run”, “better” becomes “good”, and “studies” becomes “study”. Because it understands parts of speech, lemmatization preserves the semantic meaning of words more accurately. This makes it especially useful in natural language processing tasks where language understanding and correctness are important, such as chatbots, question answering systems, and text summarization.